Universidad de La Sabana

Unidad 4: Pruebas de Integración y Automatización (CI/CD)

MAESTRÍA EN PROFUNDIZACIÓN EN INGENIERÍA DE SOFTWARE
ASIGNATURA: TESTING Y VALIDACIÓN DE SOFTWARE

Basado en la obra de: César Augusto Vega Fernández

NotebookLM

El Sistema Inmunológico del Software





Supervivencia, no lujo

En ecosistemas de despliegue masivo, las pruebas manuales son insuficientes.

La Columna Vertebral

Soporta el peso de los cambios constantes y despliegues concurrentes sin fracturarse.

El Sistema Inmunológico

Detecta problemas y protege producción antes de que el fallo sea un síntoma visible.

NotebookLM

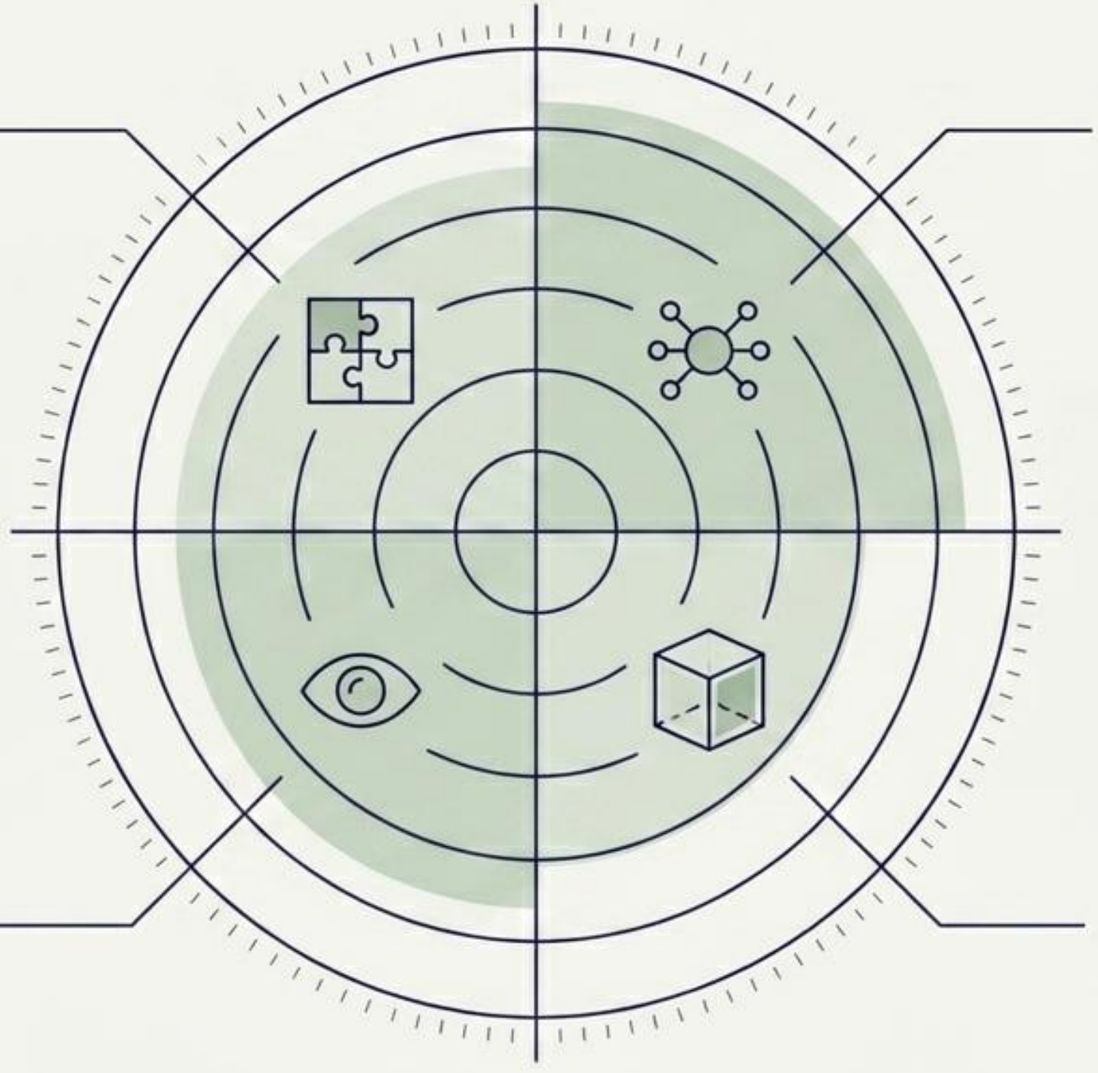
El Radar del Arquitecto: Objetivos de la Unidad

1. Pruebas Integradas

Validar la interacción entre componentes distribuidos.

2. Estrategias de API

Automatizar contratos y flujos de datos a alta velocidad.



3. Visión E2E

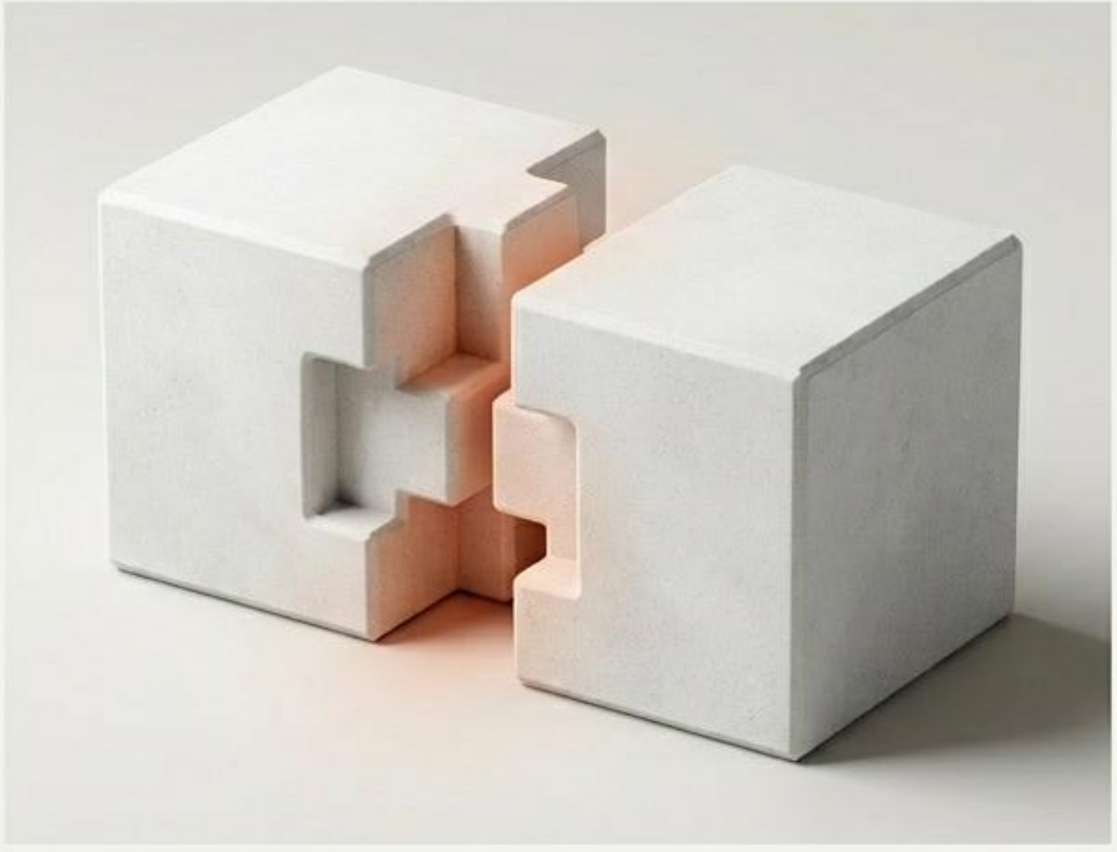
Asegurar la experiencia holística del usuario final.

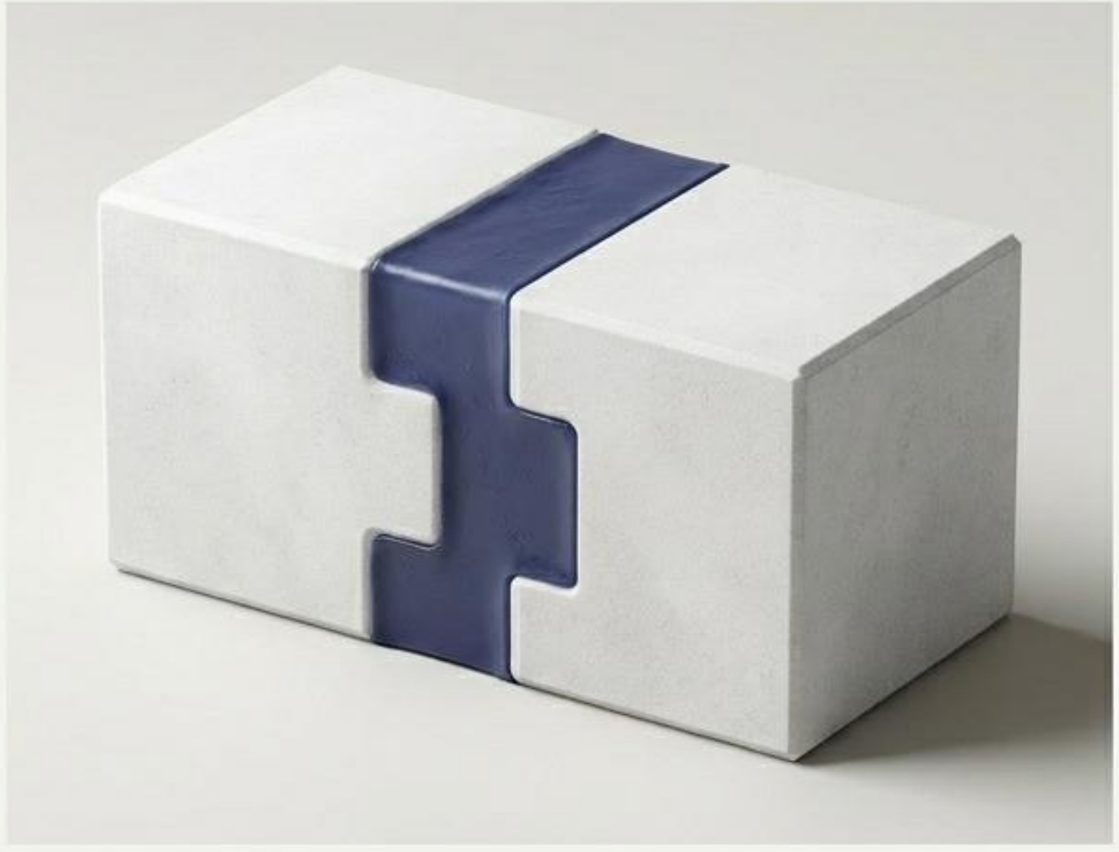
4. Simulación de Entornos

Dominar el aislamiento de dependencias y variables.

NotebookLM

El ‘Cemento’ Entre los Ladrillos





El Riesgo: Fracturas Estructurales

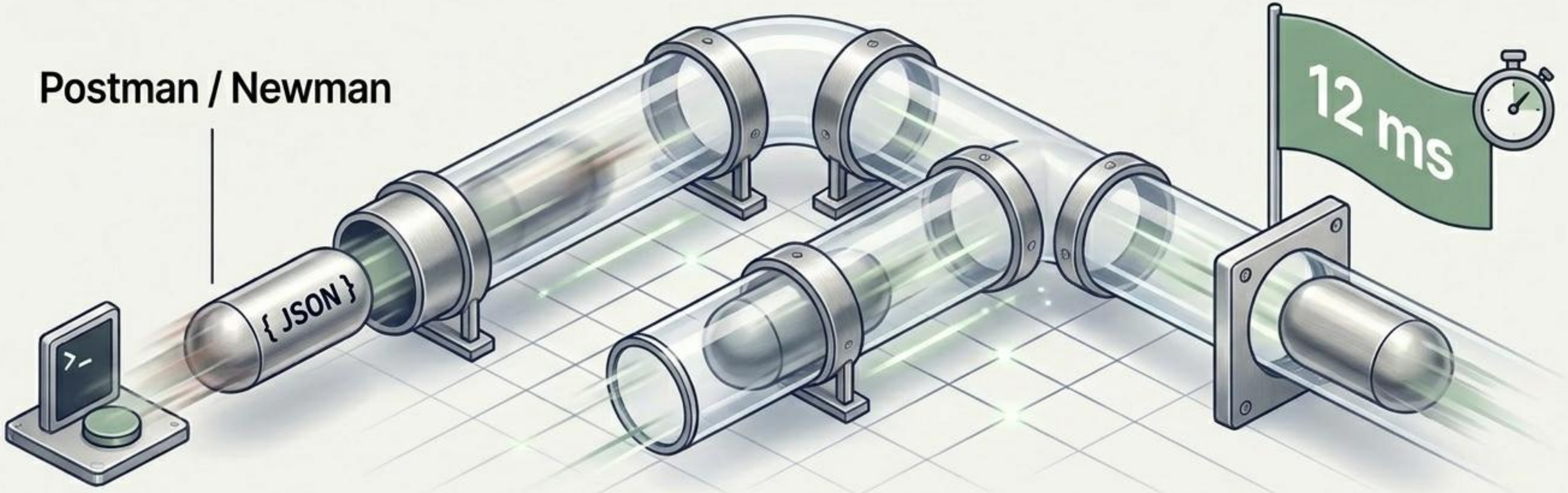
Ladrillos perfectos que no encajan. Incompatibilidad de contratos, formatos de datos desalineados o fallos de versión al unir microservicios (Ej. Pagos vs. Facturación).

El Foco: Arquitectura Distribuida

Detectar fracturas antes de ensamblar el edificio. Asegurar que las piezas individuales hablen el mismo idioma al conectarse.

NotebookLM

Automatización de APIs: Feedback Instantáneo



Velocidad DevOps

Las pruebas de API son deterministas y veloces, proporcionando rentabilidad inmediata en el pipeline.

De Colecciones a Motores CLI

Transición de ejecuciones manuales (Postman) a motores automatizados de línea de comandos (Newman).

Orquestación Sin Interfaz

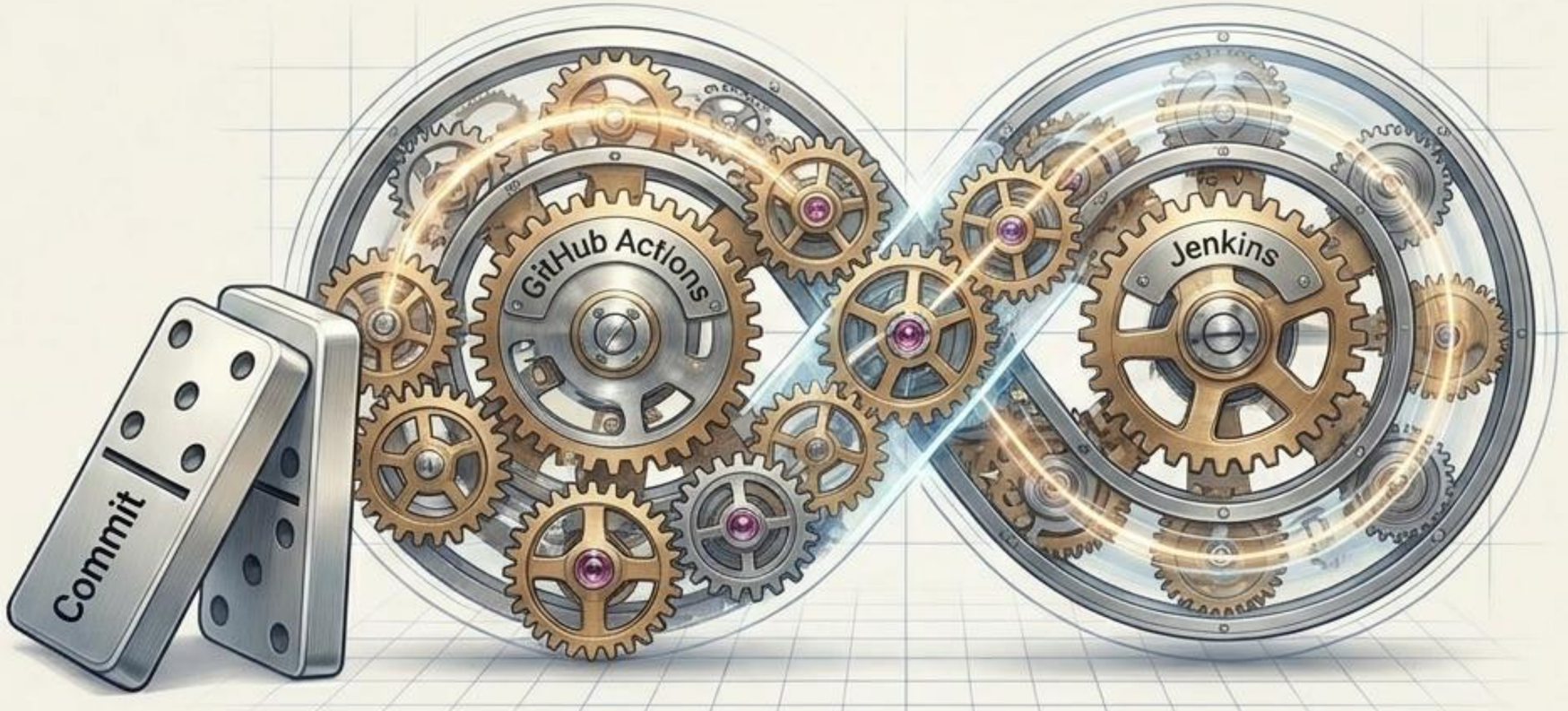
Uso de variables configurables y validaciones complejas para una ejecución orquestada sin UI.

NotebookLM

Selenium		Cypress	
Arquitectura:	WebDriver externo multiplataforma.	Arquitectura:	Inyección directa en el ciclo de vida del navegador.
Fortaleza:	Simulación precisa de acciones humanas (clics, scroll).	Fortaleza:	Timelapse visual (viaje en el tiempo para debugging rápido).
Contexto:	Versatilidad histórica y pruebas multi-navegador en sistemas heredados.	Contexto:	Diseñado nativamente para el ecosistema frontend moderno (React, Angular, Vue).

NotebookLM

Integración Continua (CI): Orquestación por Eventos



El Motor

Automatización basada en eventos. Nadie presiona 'Play'; los pull requests desencadenan el flujo entero.

Jenkins

Máxima configurabilidad, orquestación profunda e infinidad de plugins empresariales.

GitHub Actions

CI nativo, flujos desencadenados por el código directamente en el repositorio usando YAML.

El Pipeline automatizado es el único camino autorizado hacia Producción.

NotebookLM

Simulación de Entornos: Aislamiento Quirúrgico



El Problema

La fragilidad de depender de terceros: servidores caídos, latencia o APIs externas con costo por petición.

La Solución

Frameworks de simulación (ej. Mockito) para aislamiento en memoria durante el pipeline.

Diferencia Arquitectónica

- **Stub:** Proporciona respuestas preparadas (datos "quemados") para el test.
- **Mock:** Verifica el comportamiento (cómo y cuántas veces el código interactuó con la dependencia).

NotebookLM

Quality Gates: Los Guardianes del Pipeline



1. Métricas Innegociables

Semáforos de calidad objetivos: cobertura de código, severidad de defectos y tiempos de respuesta.

2. Trazabilidad Total

Cada defecto o script de prueba está vinculado unívocamente a una historia de usuario.

Sin Quality Gates estrictos, un pipeline veloz simplemente despliega desastres más rápido.

NotebookLM

El Rol del Arquitecto de Calidad

Ya no buscamos errores al final del ciclo; diseñamos la confianza desde el origen.





1. Diseñador de Riesgos: Define qué probar y con qué profundidad según el impacto en el negocio.

2. Ingeniero de Infraestructura: Construye frameworks de automatización robustos.

3. Mentor de Cultura: Guía al equipo en prácticas de Clean Code y TDD.

NotebookLM

Autopsia de un Desastre: El Pipeline que Mentía



El Síntoma

Ejecuciones locales y en CI perfectas, seguidas de un fallo catastrófico en producción.

El Diagnóstico: Falsos Negativos

Pruebas inestables (**flaky tests**), dependencias de datos que mutan sin control y entornos no homologados.

La Consecuencia Fatal: La pérdida absoluta de confianza. Si el pipeline miente, los desarrolladores ignoran el CI/CD.

NotebookLM

Antipatrones: Errores Comunes en CI/CD



Automatizar la Subjetividad

Intentar medir UX, estética visual o accesibilidad pura con scripts rígidos siempre resulta en pruebas frágiles.



Pipelines de Larga Duración

Conjuntos de pruebas monolíticas que tardan horas en ejecutarse, aniquilando por completo la agilidad del equipo DevOps.

La Regla de Oro: Automatiza lo repetitivo, evalúa manualmente lo subjetivo.

NotebookLM

El Paradigma Preventivo: Shift Left Testing



Shift Left

Validar desde el primer día (refinamiento y TDD). El costo de reparar un bug crece exponencialmente cuanto más a la derecha se detecte.

Modularidad Estricta

Crear scripts de prueba pequeños, con un propósito único, deterministas y fáciles de reemplazar.

Independencia Total

Una prueba nunca debe requerir que otra se ejecute primero para preparar su estado o sus datos.

NotebookLM



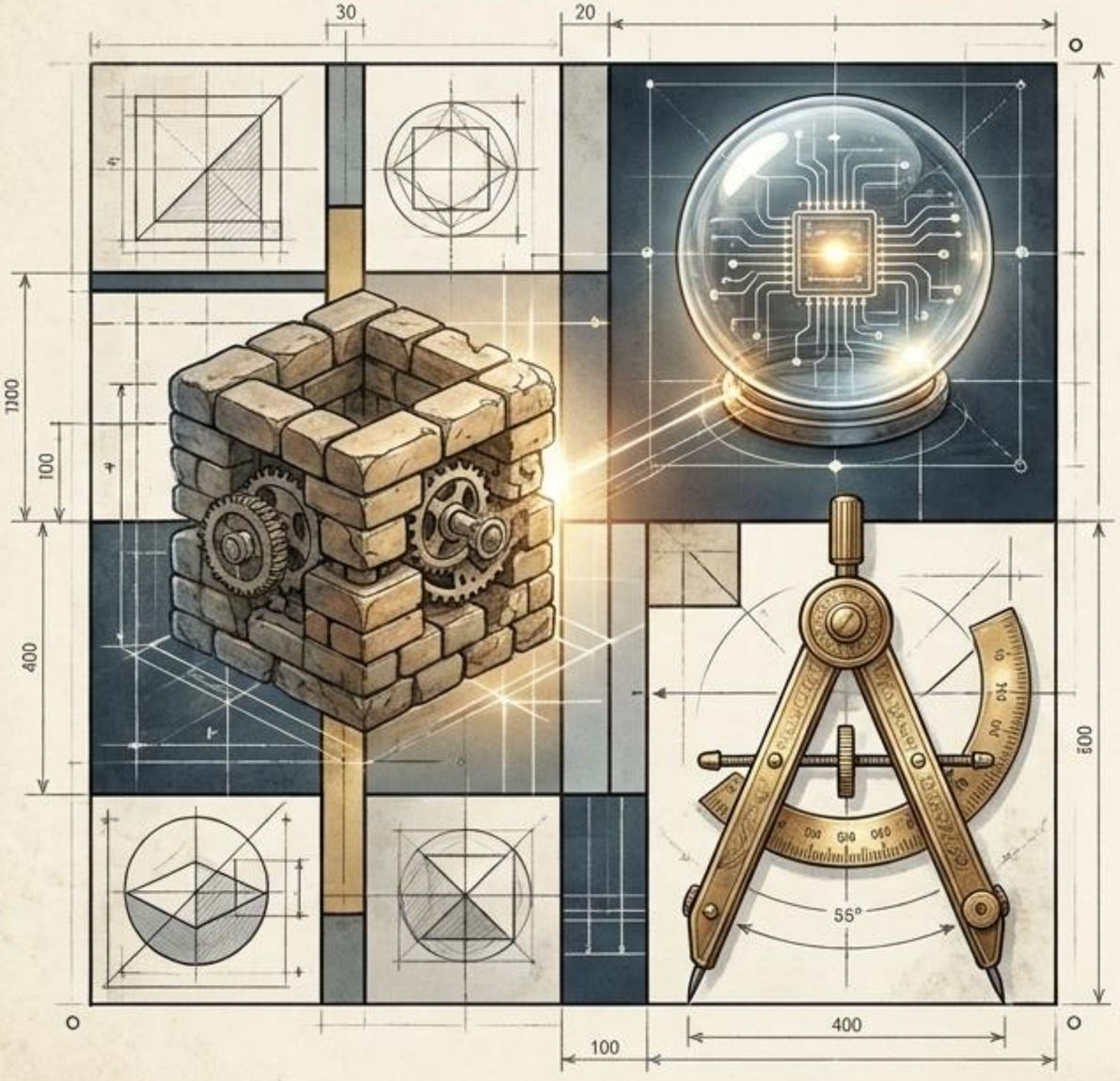
¿Confiarías en un despliegue 100% automático?

Sin humanos. Sin aprobaciones manuales. Solo código desencadenando código hasta los clientes reales.

Si la respuesta honesta es no, tu automatización aún no es un sistema inmunológico maduro. Solo es una alarma glorificada.

NotebookLM

Conclusiones: Diseñando Ecosistemas Confiables



1. La Confianza es el Fin

Las herramientas (Cypress, Jenkins, Mockito) son medios desechables; la seguridad matemática en el despliegue es el objetivo real.

2. Aislamiento y Objetividad

CI/CD no sobrevive sin aislar el caos externo (Mocking) y medir sin ninguna subjetividad (Quality Gates).

3. Construcción, no Inspección

En arquitecturas distribuidas, la calidad no se audita al final; se construye iterativamente desde el bloque unitario hasta la visión End-to-End.

NotebookLM